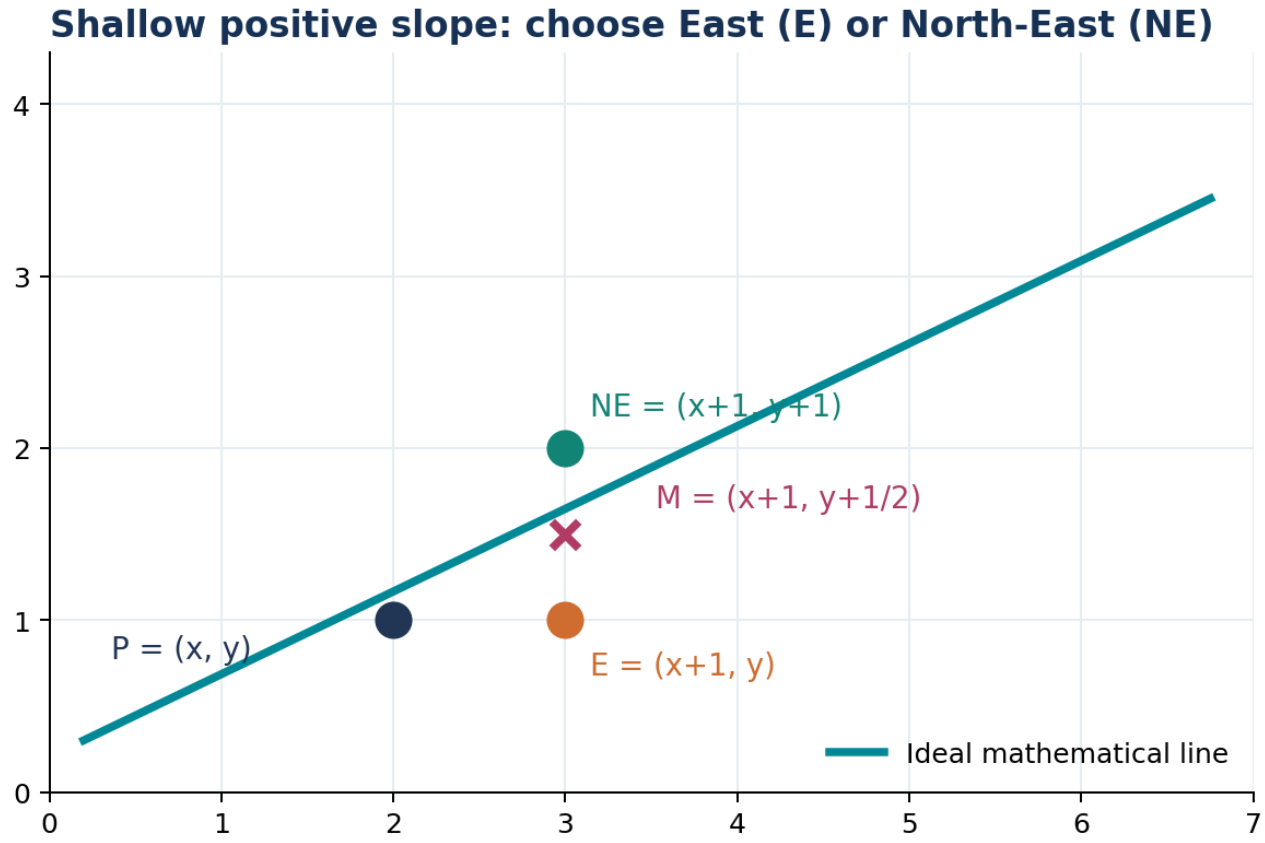
FALL 2026–2027 • THEORY + LAB

Bresenham's Midpoint Line Drawing Algorithm

Complete step-by-step instructor handout | Derivation • Decisions • Solved mathematics • Pseudocode • C++/OpenGL



Learning objectives

- Explain why raster screens need line drawing algorithms and how the midpoint test chooses a pixel.
- Derive the initial decision value $d_0 = 2\Delta y - \Delta x$ and both E/NE update rules for $0 \le m \le 1$.
- Solve shallow- and steep-slope numerical problems with an iteration table.
- Implement the algorithm in C++ using OpenGL/FreeGLUT; recognize all-octant extensions.

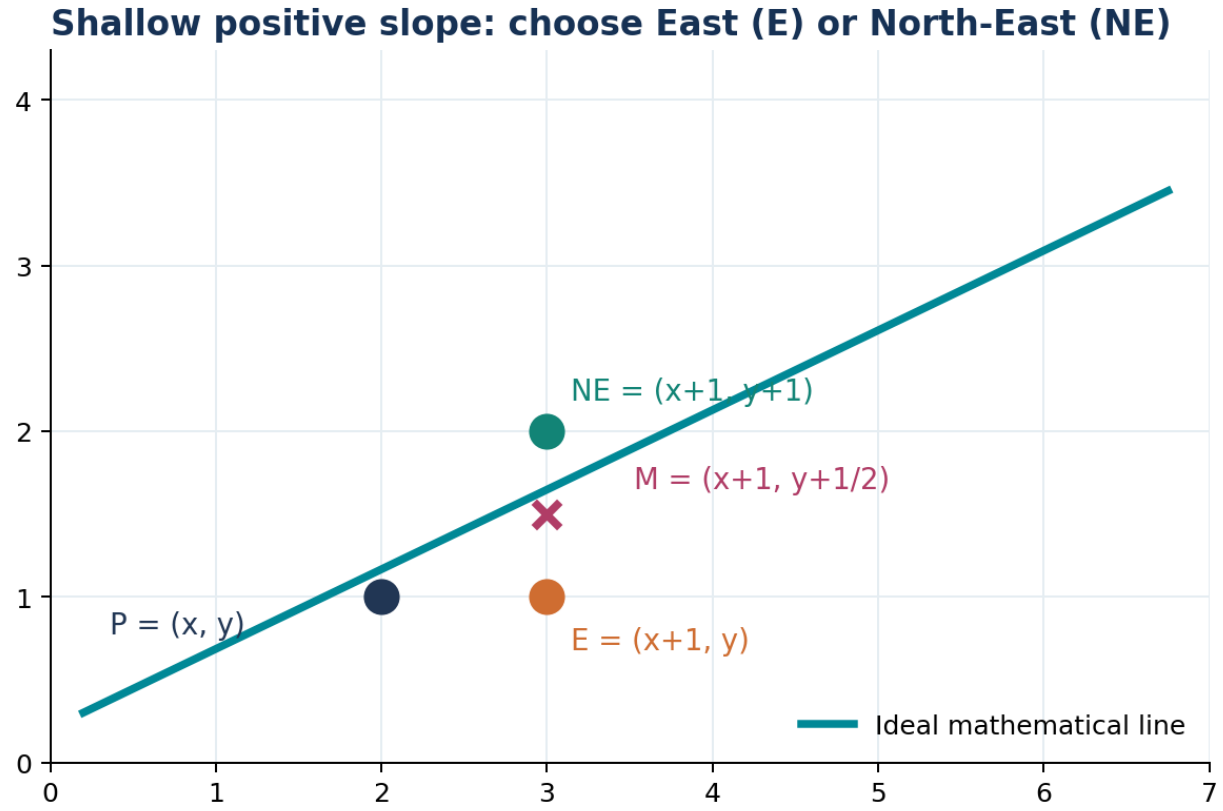
TEACHING NOTE *Begin by showing the two candidate pixels and the midpoint. Only introduce equations after students understand the geometric choice.*

1 Background: pixels, slope, and incremental decisions

A straight line consists of infinitely many continuous points, but the display is a discrete grid of pixel centres. Rasterization chooses which discrete pixels best represent the ideal geometric line.

$$\Delta x = x_2 - x_1 \qquad \Delta y = y_2 - y_1 \qquad m = \Delta y / \Delta x \quad (\text{when } \Delta x \neq 0)$$

For a shallow positive slope, 0 ≤ m ≤ 1, advance one column in x at every iteration. The next pixel must be either East (E) or North-East (NE).



$$E = (x + 1, y) \qquad NE = (x + 1, y + 1)$$

$$M = (x + 1, y + 1/2)$$

If the ideal line passes above midpoint M, choose NE; if below M, choose E. If it passes exactly through M, either candidate is equally close, so the implementation requires a consistent tie rule. The supplied slides choose E when the decision is zero.

> ***TEACHING NOTE*** *Do not claim Bresenham measures the full Euclidean distance to each candidate. For this case it compares an implicit line function at their midpoint.*

2 Derive the implicit line function

Starting with the equation used in the slides:

$$y = mx + B \quad \text{and} \quad m = \Delta y / \Delta x$$

Substitute the slope, multiply both sides by Δx, then bring every term to the left:

$$y = (\Delta y/\Delta x)x + B$$

$$\Delta x \cdot y = \Delta y \cdot x + B\Delta x$$

$$\Delta y \cdot x - \Delta x \cdot y + B\Delta x = 0$$

Write the result as F(x,y) = ax + by + c, where:

$$a = \Delta y \qquad b = -\Delta x \qquad c = B\Delta x$$

Thus F(x,y) = Δy·x − Δx·y + BΔx. Every exact point of the ideal line satisfies F(x,y) = 0. Its sign identifies which side of the line contains the midpoint.

3 Derive the initial decision parameter (shallow slope)

At the starting point $P_o=(x_o,y_o)$, the first decision is evaluated at the midpoint between the E and NE candidate pixels:

$$M_0 = (x_0 + 1, y_0 + 1/2)$$

$$d_0(\text{unscaled}) = F(M_0)$$

$$= a(x_0 + 1) + b(y_0 + 1/2) + c$$

$$= (ax_0 + by_0 + c) + a + b/2$$

Because P_o lies on the mathematical line:

$$F(x_0,y_0) = ax_0 + by_0 + c = 0$$

$$d_0(\text{unscaled}) = a + b/2 = \Delta y - \Delta x/2$$

Why multiply everything by 2?

The sign—not the magnitude—determines the selected pixel. Multiplying by positive 2 preserves the sign and eliminates the one-half, giving an entirely integer-valued initial decision for integer endpoints.

$$2 \times (\Delta y - \Delta x/2) = 2\Delta y - \Delta x$$

$$\text{FINAL SCALED START VALUE:} \quad d_0 = 2\Delta y - \Delta x$$

TEACHING NOTE *This is the "multiplying part" to emphasize: 2 is a positive scaling factor that removes 1/2. It does NOT mean the midpoint position itself changes.*

4 Derive the E update

When E is chosen, the current pixel moves from (x,y) to (x+1,y); the next midpoint becomes (x+2, y+1/2).

$$dnew - dold = F(x+2, y+1/2) - F(x+1, y+1/2)$$

$$= a = \Delta y \quad (\text{unscaled})$$

Because the stored parameter is twice the unscaled one:

$$\text{IF E IS CHOSEN:} \quad dnew = dold + 2\Delta y$$

$$\text{Coordinates: } (x,y) \rightarrow (x+1, y)$$

5 Derive the NE update

When NE is chosen, the current pixel moves from (x,y) to (x+1,y+1), so the next midpoint becomes (x+2, y+3/2).

$$dnew - dold = F(x+2, y+3/2) - F(x+1, y+1/2)$$

$$= a + b = \Delta y - \Delta x \quad (\text{unscaled})$$

$$\text{IF NE IS CHOSEN:} \quad d_{new} = d_{old} + 2(\Delta y - \Delta x)$$

$$\text{Coordinates: } (x,y) \rightarrow (x+1, y+1)$$

6 Core algorithm: $0 \le m \le 1$ and $x_2 > x_1$

This is the original positive, shallow-slope case taught in the supplied slides. Define $\Delta x = x_2 - x_1 \ge 0$, $\Delta y = y_2 - y_1 \ge 0$, with $\Delta y \le \Delta x$.

Decision	Pixel chosen	Coordinates after move	Update d
d > 0	NE	x ← x+1; y ← y+1	d ← d + 2(Δy − Δx)
d ≤ 0	E	x ← x+1; y unchanged	d ← d + 2Δy

1. Calculate Δx and Δy, checking that the assumptions hold.
2. Initialize $x=x_1$, $y=y_1$, and $d=2\Delta y-\Delta x$.
3. Plot the initial pixel (x_1,y_1).
4. For each of the next Δx columns, inspect the current d.
5. Choose NE if d>0; otherwise choose E. Update coordinates and d.
6. Plot the new pixel. Stop when the final endpoint is reached.

Language-independent pseudocode

```
BRESENHAM_SHALLOW(x1, y1, x2, y2)
    dx = x2 - x1
    dy = y2 - y1
    // Preconditions: dx > 0 and 0 <= dy <= dx
    d = 2*dy - dx
    x = x1; y = y1
    PLOT(x, y)
    while x < x2
        if d > 0
            x = x + 1; y = y + 1
            d = d + 2*(dy - dx)   // NE
        else
            x = x + 1
            d = d + 2*dy          // E
        PLOT(x, y)
```

Three facts students should memorize

$$\text{Start d: } 2\Delta y - \Delta x$$

$$\text{E: } \quad d + 2\Delta y$$

$$\text{NE: } \quad d + 2(\Delta y - \Delta x)$$

7 Worked example from the lecture: (4,5) → (7,7)

First compute Δx=7−4=3 and Δy=7−5=2, so m=2/3 (shallow).

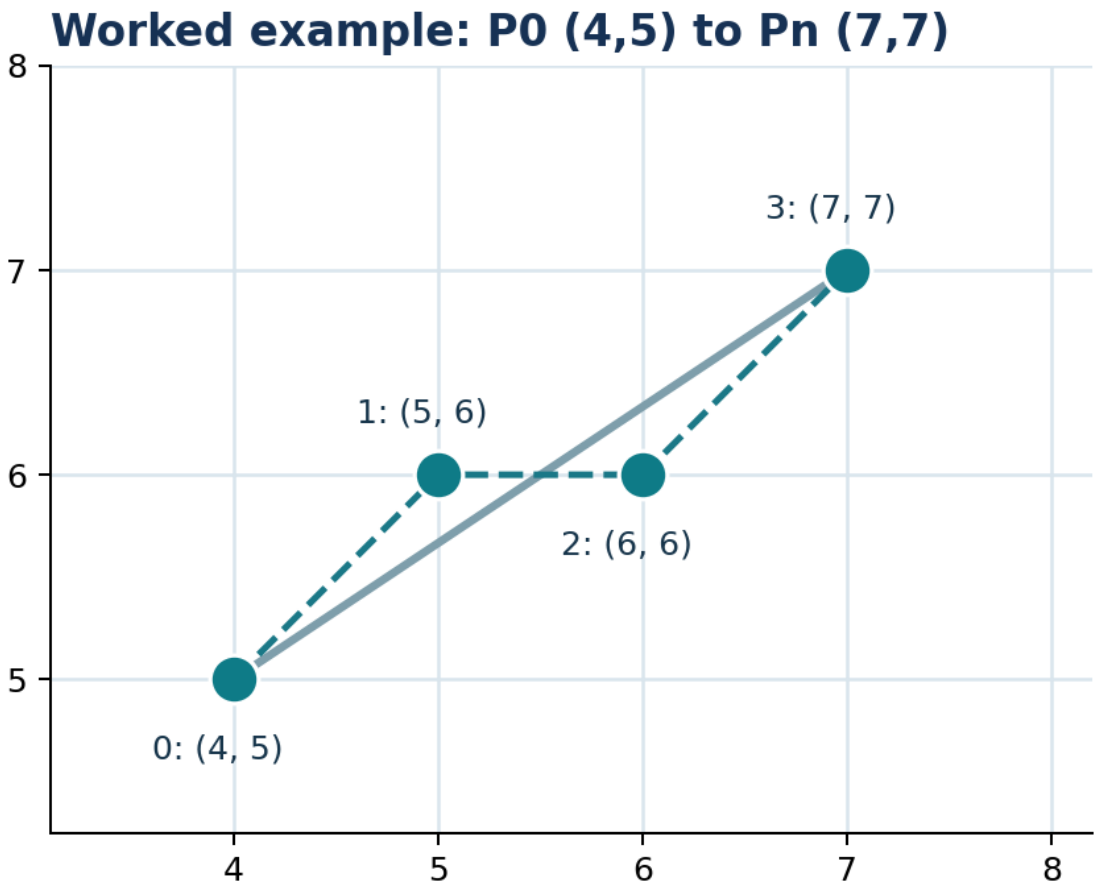
$$d_0 = 2(2) - 3 = 1$$

$$\text{Straight E increment} = 2\Delta y = +4$$

$$\text{Diagonal NE increment} = 2(\Delta y - \Delta x) = 2(2-3) = -2$$

From	d before	Choice	To / plotted pixel	d after
(4,5)	1 (>0)	NE	(5,6)	1−2 = −1
(5,6)	−1 (≤0)	E	(6,6)	−1+4 = 3

(6,6)	3 (>0)	NE	(7,7)	endpoint



PIXEL SEQUENCE: (4,5), (5,6), (6,6), (7,7)

8 Longer shallow-slope example, including a tie

Example: draw the line from (2,3) to (10,7). This is an added instructor practice problem, not the lecture slide example.

$\Delta x = 10 - 2 = 8 \quad \Delta y = 7 - 3 = 4 \quad m = 4/8 = 0.5$

$d_0 = 2(4) - 8 = 0$ E increment: $+8$ NE increment: -8

The source deck uses d ≤ 0 ⇒ E. Hence, the first decision at d=0 selects E.

Step	Current pixel	d before	Decision	Next pixel	d after
1	(2,3)	0	E	(3,3)	8
2	(3,3)	8	NE	(4,4)	0
3	(4,4)	0	E	(5,4)	8
4	(5,4)	8	NE	(6,5)	0
5	(6,5)	0	E	(7,5)	8
6	(7,5)	8	NE	(8,6)	0
7	(8,6)	0	E	(9,6)	8
8	(9,6)	8	NE	(10,7)	end

PIXELS: (2,3), (3,3), (4,4), (5,4), (6,5), (7,5), (8,6), (9,6), (10,7)

TEACHING NOTE *Ask students why two correct rasterizers may disagree on a tie. It is the documented tie policy, not necessarily a faulty derivation.*

9 Steep positive slope: m > 1

When Δy > Δx ≥ 0, advance by one row each time. The candidates are N=(x,y+1) and NE=(x+1,y+1). The source deck states the following integer decision rules.

$$\text{INITIAL: } d_0 = 2\Delta x - \Delta y$$

Decision	Chosen pixel	New coordinates	Update d
d > 0	NE	x ← x+1; y ← y+1	d ← d + 2(Δx − Δy)
d ≤ 0	N	x unchanged; y ← y+1	d ← d + 2Δx

Why the formula swaps Δx and Δy: y is now the dominant coordinate, so the same midpoint reasoning is applied after exchanging the horizontal and vertical axes.

Full mathematical derivation for the steep case

For a steep, rising line, define an implicit function with the sign chosen so that positive selects NE:

$$G(x,y) = \Delta x(y - y_1) - \Delta y(x - x_1)$$

At a current pixel (x,y), N=(x,y+1) and NE=(x+1,y+1). Their midpoint is (x+1/2,y+1). At the initial endpoint, $G(x_1,y_1)=0$:

$$d_0(\text{unscaled}) = G(x_1 + 1/2,\ y_1 + 1) = \Delta x - \Delta y/2$$

$$\text{Multiply by positive 2: } d_0 = 2\Delta x - \Delta y$$

If N is selected, the next midpoint moves up one row; G changes by +Δx. If NE is selected, that next midpoint moves one column right and one row up; G changes by (Δx−Δy). Scale both changes by 2:

$$\text{N: } d_{new} = d_{old} + 2\Delta x$$

$$\text{NE: } d_{new} = d_{old} + 2(\Delta x - \Delta y)$$

10 Fully solved steep-slope example

Draw the line from (2,2) to (4,7). This is an instructor expansion of the source lecture's steep-slope rules.

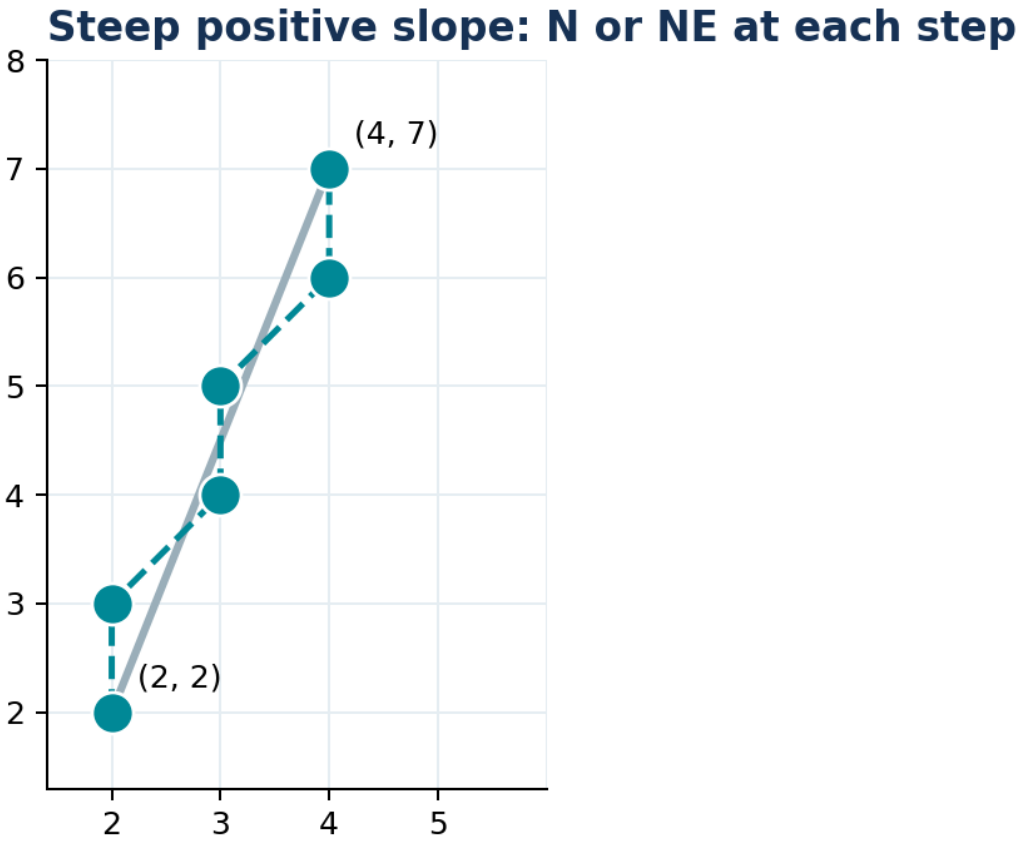
$$\Delta x = 4 - 2 = 2 \qquad \Delta y = 7 - 2 = 5 \qquad m = 5/2 = 2.5 > 1$$

$$d_0 = 2(2) - 5 = -1$$

$$\text{N increment} = +2\Delta x = +4$$

$$\text{NE increment} = 2(\Delta x - \Delta y) = 2(2 - 5) = -6$$

Step	From	d before	Choice	Next pixel	d after
1	(2,2)	−1	N	(2,3)	−1+4=3
2	(2,3)	3	NE	(3,4)	3−6=−3
3	(3,4)	−3	N	(3,5)	−3+4=1
4	(3,5)	1	NE	(4,6)	1−6=−5
5	(4,6)	−5	N	(4,7)	end



PIXELS: (2,2), (2,3), (3,4), (3,5), (4,6), (4,7)

Steep pseudocode

```
BRESENHAM_STEEP(x1, y1, x2, y2)
    dx = x2 - x1; dy = y2 - y1
    // Preconditions: dy > dx >= 0
    d = 2*dx - dy
    x = x1; y = y1
    PLOT(x, y)
    while y < y2
        if d > 0
            x = x + 1; y = y + 1
            d = d + 2*(dx - dy)  // NE
        else
            y = y + 1
            d = d + 2*dx         // N
        PLOT(x, y)
```

11 Summary: identify the correct rule before calculating

Case	Driving axis	Candidate pixels	Start d	If d > 0	If d ≤ 0
0 ≤ m ≤ 1	x	E, NE	2Δy−Δx	NE; +2(Δy−Δx)	E; +2Δy
m > 1	y	N, NE	2Δx−Δy	NE; +2(Δx−Δy)	N; +2Δx

Special cases: horizontal lines (Δy=0) advance only x; vertical lines (Δx=0) advance only y; a 45° positive diagonal (Δx=Δy) advances both. The formulas above assume the line moves rightward and upward.

Which information comes from the slide deck?

Slides 24–44 derive and demonstrate the midpoint decision algorithm; slides 47–48 summarize the two positive-slope cases. The next section adds general handling for negative slopes and reversed endpoints as a programming extension.

12 Instructor expansion: all directions / all octants

The same integer error principle works for negative slopes, reversed endpoints and all eight octants. Use absolute magnitudes of Δx and Δy plus signed direction steps. This is standard implementation knowledge added for your lab rather than a line-by-line reproduction of the source slides.

$$dx = |x_2 - x_1| \qquad dy = |y_2 - y_1|$$

$$sx = +1 \text{ if } x_1 < x_2, \text{ otherwise } -1$$

$$sy = +1 \text{ if } y_1 < y_2, \text{ otherwise } -1$$

$$error = dx - dy$$

At each plotted pixel, compute e2=2·error before modifying either coordinate. Depending on two comparisons, update x and/or y. The conditions are independent so diagonal moves can update both coordinates.

```
ALL_OCTANTS(x1, y1, x2, y2)
    dx = abs(x2 - x1); dy = abs(y2 - y1)
    sx = (x1 < x2) ? +1 : -1
    sy = (y1 < y2) ? +1 : -1
    err = dx - dy
    loop
        PLOT(x1, y1)
        if x1 == x2 AND y1 == y2: stop
        e2 = 2*err
        if e2 > -dy: err = err - dy; x1 = x1 + sx
        if e2 <  dx: err = err + dx; y1 = y1 + sy
```

TEACHING NOTE *Check the source's tie rule separately when comparing implementations. Tie conventions can change a few selected pixels while leaving the line rasterization valid.*

13 Complete C++ / OpenGL-FreeGLUT program

This standalone code draws four lines in different orientations using the all-octants version. It uses the compatibility OpenGL interface used in your existing lab materials and can be compiled with FreeGLUT configured in Code::Blocks.

```
#include <GL/glut.h>
#include <cstdlib>   // abs

void bresenham(int x1, int y1, int x2, int y2)
{
    int dx = abs(x2 - x1);
    int dy = abs(y2 - y1);
    int sx = (x1 < x2) ? 1 : -1;
    int sy = (y1 < y2) ? 1 : -1;
    int err = dx - dy;

    glBegin(GL_POINTS);
    while (true)
    {
        glVertex2i(x1, y1); // Plot the current integer pixel
        if (x1 == x2 && y1 == y2) break;

        int e2 = 2 * err;
        if (e2 > -dy)
        {
            err -= dy;
            x1 += sx;
        }
        if (e2 < dx)
```

```
        {
            err += dx;
            y1 += sy;
        }
    }
    glEnd();
}

void display()
{
    glClear(GL_COLOR_BUFFER_BIT);
    glPointSize(3.0f);

    glColor3f(0.1f, 0.4f, 0.9f);  // Shallow positive
    bresenham(50, 60, 340, 190);

    glColor3f(0.0f, 0.6f, 0.35f); // Steep positive
    bresenham(420, 40, 520, 320);

    glColor3f(0.9f, 0.35f, 0.2f); // Negative slope
    bresenham(50, 340, 330, 230);

    glColor3f(0.4f, 0.25f, 0.7f); // Reversed endpoints
    bresenham(720, 340, 550, 150);
    glFlush();
}

void init()
{
    glClearColor(1.0f, 1.0f, 1.0f, 1.0f);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(0, 800, 0, 400);
}

int main(int argc, char** argv)
{
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize(900, 500);
    glutCreateWindow("Bresenham - All Octants");
    init();
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}
```

Lab reminder: OpenGL here is only plotting the discrete pixels returned by the algorithm; GL_LINES is not used, because the educational goal is to generate those pixels yourself.

14 Classroom delivery guide

Time	What to teach	Activity / check
0–10 min	Real line vs raster pixel grid	Sketch a diagonal on squared paper
10–20 min	E and NE, midpoint M	Students point to closest candidate
20–40 min	Line equation and F(x,y)	Convert y=mx+B into implicit form
40–55 min	Initial d_o; scaling by 2	Ask why the sign is unchanged
55–70 min	Derive E and NE updates	Cancel terms step by step
70–85 min	Solve slide example (4,5)→(7,7)	Students complete decision table
85–100 min	Steep case and example	Show N/NE and swapped axes
100–120 min	Code + discussion	Modify endpoints and predict pixels

Common mistakes and how to correct them

- Incorrect slope: m = Δy/Δx, not Δx/Δy. Check whether the line is shallow or steep before choosing a table.

- Confusing “candidate” and “plotted pixel”: the current d decides the next pixel, then d updates for the next iteration.
- Updating d before choosing: use the current d for the current decision and only afterward apply the corresponding increment.
- Forgetting the scaled parameter: once d has been multiplied by 2, the update must also use 2Δy or 2(Δy−Δx).
- Making the wrong tie decision: this lecture uses d≤0 for E (shallow) or N (steep).
- Using shallow positive-slope code on a negative or steep line: either transform the octant or use the generalized implementation.
- Plotting too few points: include both starting and ending endpoints.

15 Short questions / oral viva

- Why is Bresenham an incremental line algorithm?
- What do E and NE represent?
- What is the midpoint and why test it?
- How is F(x,y)=ax+by+c related to y=mx+B?
- Why is multiplying the decision expression by 2 valid?
- For 0≤m≤1, what do d>0 and d≤0 mean?
- Where do the +2Δy and +2(Δy−Δx) updates come from?
- What changes when slope exceeds 1?
- Can different tie-breaking rules produce different valid raster patterns?
- How would you handle a negative slope in code?

16 Student practice problems

Require the full board-style solution: calculate Δx, Δy and m; choose the case; find initial d; precompute both updates; show every chosen pixel in a table; identify the final pixel sequence.

Problem	Endpoints	Case to recognize
A	(1,2) to (7,5)	Shallow, positive
B	(3,1) to (6,9)	Steep, positive
C	(2,4) to (10,4)	Horizontal special case
D	(5,2) to (5,8)	Vertical special case
E	(2,8) to (9,3)	Negative slope: extension

Instructor answer check — Problem A

Δx=6, Δy=3, m=0.5. Initial d=2(3)−6=0. E increment +6; NE increment 2(3−6)=−6. Under the slide convention, ties choose E.

From	d	Move	Plot	Next d
(1,2)	0	E	(2,2)	6
(2,2)	6	NE	(3,3)	0
(3,3)	0	E	(4,3)	6
(4,3)	6	NE	(5,4)	0
(5,4)	0	E	(6,4)	6
(6,4)	6	NE	(7,5)	end

Instructor answer check — Problem B

Δx=3, Δy=8, m=8/3 > 1. Initial d=6−8=−2; N update +6; NE update 2(3−8)=−10.

From	d	Move	Plot	Next d
(3,1)	−2	N	(3,2)	4
(3,2)	4	NE	(4,3)	−6
(4,3)	−6	N	(4,4)	0
(4,4)	0	N	(4,5)	6
(4,5)	6	NE	(5,6)	−4
(5,6)	−4	N	(5,7)	2
(5,7)	2	NE	(6,8)	−8
(6,8)	−8	N	(6,9)	end